

Sistema Integrado de Atendimento e Execução de Serviços

Tech Challenge — Fase 1 · Pós-graduação FIAP em Arquitetura de Software (SOAT)

Identificação

Nome do grupo: Humberto Vitalino (projeto individual)

Participantes:

Nome	Username Discord
Humberto Vitalino	hbrto.

Links

Repositório (GitHub): <https://github.com/HumbertoVitalino/fiap-postgraduated-workshop>

Repositório público — visibilidade definida a pedido da coordenação do curso, para facilitar a avaliação.

Documentação DDD (Miro): [Board no Miro](#)

Contém o Event Storming completo dos fluxos de criação/acompanhamento de OS e de gestão de peças e insumos, diagramas de DDD e a Linguagem Ubíqua aplicada ao domínio da oficina.

Resumo da solução entregue

MVP do back-end de um sistema integrado de atendimento e execução de serviços para uma oficina mecânica, construído em .NET 10 com Clean Architecture (Domain → Application → Infrastructure → Api) e Domain-Driven Design.

- **Gestão administrativa:** CRUD completo de Clientes, Veículos, Serviços (catálogo) e Peças/Insumos (com controle de estoque).
- **Ordem de Serviço:** máquina de estados completa — Recebida → Em diagnóstico → Aguardando aprovação → (Em execução → Finalizada | Cancelada) → Entregue — com orçamento calculado automaticamente, reserva/baixa de estoque e histórico de status.
- **Acompanhamento público:** consulta anônima da OS pelo cliente via CPF/CNPJ + placa, sem autenticação e sem vazar existência de cadastro.
- **Segurança:** autenticação JWT com policies por papel (Admin/Attendant/Mechanic), hashing de senha com BCrypt, validação de CPF/CNPJ e placa de veículo.

- **Qualidade:** 209 testes automatizados (unitários + integração contra SQL Server real via Docker), 97,3% de cobertura de linhas em Domain/Application (SonarQube Cloud), pipeline de CI com build, testes, análise SonarQube e CodeQL (SAST) a cada push/PR.
- **Infraestrutura:** Dockerfile + docker-compose orquestrando API, banco (schema via script de init) e dado de demonstração seedado automaticamente.

Relatório de análise de vulnerabilidades

Ferramentas e processo

A análise estática de segurança (SAST) roda continuamente no pipeline de CI, sem depender de execução manual:

- **CodeQL** (GitHub Advanced Security), com o conjunto de queries `security-and-quality` — executa em todo Pull Request para a branch `release` e semanalmente (agendado).
- **SonarQube Cloud**, com Coverlet medindo cobertura — executa a cada push em `release`, analisando `Domain` e `Application` (os domínios críticos do desafio).

Resultado atual no SonarQube Cloud

Indicador	Resultado
Quality Gate	Passed
Security (issues abertos)	0 — nota A
Reliability (issues abertos)	0 — nota A
Maintainability (issues abertos)	14 — nota A (débito técnico leve, sem risco de segurança)
Cobertura de testes	97,3% em Domain/Application (meta do desafio: 80%)
Duplicação	1,3%

Achados do CodeQL e tratamento dado

O CodeQL identificou **15 alertas de severidade *medium***, concentrados em duas categorias, todas no fluxo de logging estruturado dos use cases da camada Application:

Regra	Ocorrências	Severidade	Descrição do risco	Status
<code>cs/log-forging</code>	9	medium	Valores fornecidos pelo usuário (e-mail no cadastro/login/atualização, código de serviço/peça) eram gravados diretamente na mensagem de log. Um valor contendo <code>\r / \n</code> poderia forjar linhas de log falsas (log injection), adulterando o histórico de auditoria.	corrigido
<code>cs/exposure-of-sensitive-information</code>	6	medium	O e-mail do usuário/cliente (dado pessoal) era gravado em texto puro nos logs de aplicação nos fluxos de criação, login e atualização.	corrigido

Correção aplicada: foi introduzido um helper

(`Fiap.Workshop.Application.Commons.LoggingExtensions`) com dois métodos de extensão:

- `SanitizeForLog()` — remove caracteres de quebra de linha (`\r / \n`) do valor antes de logar, eliminando a possibilidade de forjar entradas de log (aplicado a códigos de serviço/peça e telefone).
- `MaskEmail()` — sanitiza e mascara o e-mail (ex.: `t***@example.com`) antes de logar, resolvendo simultaneamente a exposição do dado pessoal e o risco de log forging (aplicado a todo log que hoje envolve e-mail).

A correção foi aplicada nos seis use cases apontados pelos alertas (`CreateUserUseCase` , `LoginUserUseCase` , `UpdateUserUseCase` , `UpdateCustomerUseCase` , `CreateServiceUseCase` , `CreateInventoryItemUseCase`), com a suíte de 209 testes unitários ajustada e passando integralmente. A confirmação definitiva do fechamento dos 15 alertas depende da próxima execução do CodeQL sobre a branch `release` (o CodeQL roda por Pull Request e agendamento semanal, não a cada commit).

Demais achados (sem severidade de segurança)

O CodeQL também reporta, no mesmo conjunto de queries `security-and-quality` , itens de estilo/manutenibilidade sem risco de segurança associado — `cs/useless-upcast` , `cs/useless-assignment-to-local` , `cs/missed-readonly-modifier` , `cs/nested-if-statements` , `cs/catch-of-all-exceptions` , `cs/path-combine` , `cs/constant-condition` e `cs/linq/missed-select` . Não representam vulnerabilidades e não bloquearam esta entrega; permanecem como débito técnico de qualidade para iteração futura.

Outras práticas de segurança já em vigor

- Hash de senha com BCrypt (work factor 12) — nunca senha em texto puro em repouso.
- Autenticação JWT com policies por papel de negócio (`AdminOnly` , `AttendantOnly` , `MechanicOnly`), sem role genérico de acesso amplo.
- Validação de dados sensíveis na borda da API: CPF/CNPJ (dígito verificador) e placa de veículo (padrão Mercosul e antigo), via FluentValidation.
- Tratamento centralizado de exceções de domínio (`DomainExceptionHandler`) e de erros gerais (`UseExceptionHandler`), evitando vazamento de stack trace/detalhes internos nas respostas HTTP.
- Consulta pública da OS pelo cliente desenhada para nunca revelar se um CPF/CNPJ ou placa existe no sistema quando os dados não conferem (sempre `200` com lista vazia).